

Blocco 7 - DDL, DML e transazioni

Modificare schema e dati in modo controllato

Relational Databases & SQL

30 min teoria + 90 min pratica

1. Rappresentazione iniziale

Partiamo da una soluzione semplice e concreta, prima di introdurre il modello generale.

Perché questo blocco

Problema didattico

Finora abbiamo letto dati. In un sistema reale bisogna anche creare strutture e modificare record. Un UPDATE senza filtro o una cancellazione non controllata può produrre danni immediati.

Idea guida

Prima osserviamo una rappresentazione naturale, poi ne discutiamo i limiti e solo dopo formalizziamo.

Modifica manuale di una tabella

Situazione concreta

Aggiornare un prezzo o inserire un ordine sembra semplice: si cambia una cella. In un DBMS quella modifica deve rispettare vincoli, storia e possibilità di controllo.

```
UPDATE products  
SET unit_price = unit_price * 1.05;
```

Limiti della rappresentazione iniziale

- ▶ manca un filtro sicuro;
- ▶ non sappiamo quali righe cambiano;
- ▶ non salviamo il valore precedente;
- ▶ non c'è verifica prima del commit.

2. Soluzione più generale

Riorganizziamo l'informazione in una forma più flessibile e controllabile.

Da modifica diretta a procedura controllata

Principio

DDL, DML e transazioni permettono di modificare schema e dati con un processo: selezione preventiva, modifica mirata, verifica e decisione finale.

DDL, DML e transazioni: cosa cambia

Famiglia	Domanda	Esempi
DDL	Come è fatta la struttura?	CREATE TABLE, ALTER TABLE, DROP TABLE
DML	Quali righe entrano, cambiano o escono?	INSERT, UPDATE, DELETE
TCL	Quando confermo o annullo?	BEGIN, COMMIT, ROLLBACK

Lettura operativa

DDL e DML non sono solo sintassi: sono decisioni con impatto su consistenza, dati storici e possibilità di recupero.

Perché non basta scrivere il comando

- ▶ La modifica può riguardare più righe del previsto.
- ▶ I vincoli possono bloccare inserimenti o cancellazioni.
- ▶ Il valore precedente può servire per audit o controllo.
- ▶ Una transazione separa il test dalla decisione definitiva.

Regola pratica

Ogni comando che modifica dati deve avere una query gemella di controllo.

Esempio di riorganizzazione

```
BEGIN;  
SELECT ... WHERE category = 'accessories';  
UPDATE products SET unit_price = ... WHERE category = 'accessories'  
RETURNING product_id, unit_price;  
ROLLBACK;
```

Processo di trasformazione

- ① scrivere una query di pre-controllo;
- ② aprire una transazione;
- ③ modificare solo le righe target;
- ④ usare RETURNING;
- ⑤ decidere COMMIT o ROLLBACK.

Analogia: lavori su un impianto elettrico

Analogia o caso esplicativo

Prima di intervenire si legge lo schema, si isola la zona, si misura, si modifica e si misura di nuovo. Le transazioni e le query di controllo servono alla stessa disciplina operativa.

Transazione: confine della responsabilità

Definizione operativa

Una transazione è un blocco di lavoro che deve essere confermato tutto insieme oppure annullato tutto insieme.

- ▶ BEGIN: apro il laboratorio controllato.
- ▶ Comandi DDL/DML: preparo e verifico la modifica.
- ▶ ROLLBACK: annullo il test.
- ▶ COMMIT: rendo definitiva la modifica.

ACID senza formalismo pesante

- ▶ **Atomicità:** o passa tutto o non passa nulla.
- ▶ **Consistenza:** i vincoli devono restare veri.
- ▶ **Isolamento:** le transazioni concorrenti non devono confondersi.
- ▶ **Durabilità:** dopo il commit la modifica è persistente.

Nel laboratorio

Ci concentriamo soprattutto su atomicità e consistenza: sono le proprietà che si vedono subito quando si prova ROLLBACK.

3. Formalizzazione

Diamo un nome preciso ai concetti emersi dall'esempio.

- ▶ DDL modifica la struttura;
- ▶ DML modifica i dati;
- ▶ transazione come unità atomica;
- ▶ COMMIT conferma;
- ▶ ROLLBACK annulla.

Vocabolario essenziale del blocco

- ▶ **DDL**: istruzioni che definiscono o modificano oggetti del database.
- ▶ **DML**: istruzioni che inseriscono, aggiornano o cancellano righe.
- ▶ **Transazione**: unità di lavoro confermata o annullata insieme.
- ▶ **RETURNING**: modo per vedere cosa è stato modificato.
- ▶ **Rollback**: annullamento della transazione.

Anatomia di una tabella

- ▶ una colonna ha nome, tipo e opzionalmente vincoli;
- ▶ la chiave primaria identifica una riga;
- ▶ la chiave esterna collega due tabelle;
- ▶ NOT NULL, UNIQUE e CHECK riducono stati impossibili;
- ▶ i default evitano di ripetere valori standard.

Esempio DDL: tabella di audit minima

```
CREATE TABLE product_price_audit (  
    audit_id bigserial PRIMARY KEY,  
    product_id integer NOT NULL REFERENCES products(product_id),  
    old_price numeric(10, 2) NOT NULL,  
    new_price numeric(10, 2) NOT NULL,  
    changed_at timestamp NOT NULL DEFAULT now(),  
    reason text NOT NULL  
);
```

La tabella non memorizza solo il nuovo prezzo: conserva anche contesto e tracciabilità.

Come riconoscere un buon DDL

- ▶ i tipi rappresentano il dominio dei valori;
- ▶ i vincoli impediscono dati senza significato;
- ▶ le chiavi rendono esplicite identità e collegamenti;
- ▶ i nomi aiutano a leggere il modello senza commenti esterni;
- ▶ una tabella di log contiene anche motivo, data e riferimento all'oggetto cambiato.

Come riconoscere una buona soluzione

- ▶ la modifica è preceduta da una query di selezione equivalente;
- ▶ il numero di righe modificate è atteso;
- ▶ la transazione termina con una decisione esplicita;
- ▶ gli effetti sono verificati con query successive.

4. Esempio guidato

Applichiamo il modello generale a un caso piccolo, leggendo ogni passaggio.

Esempio: situazione iniziale

Domanda o frase di dominio

Caso: aumentare del 5% il prezzo dei prodotti accessori attivi, registrando vecchio prezzo, nuovo prezzo e motivo della modifica.

Esempio: ragionamento

- ▶ il filtro deve identificare categoria e prodotti attivi;
- ▶ prima si contano le righe interessate;
- ▶ l'audit salva vecchio e nuovo prezzo;
- ▶ la transazione consente revisione prima del commit.

Esempio: pre-controllo delle righe candidate

```
SELECT product_id, sku, unit_price
FROM products
WHERE category = 'accessories'
      AND active = true
ORDER BY product_id;
```

Questa query deve restituire esattamente le righe che l'UPDATE modificherà.

Esempio: aggiornamento controllato

```
BEGIN;  
  
UPDATE products  
SET unit_price = round(unit_price * 1.05, 2)  
WHERE category = 'accessories'  
    AND active = true  
RETURNING product_id, sku, unit_price;  
  
ROLLBACK;
```

Esempio: inserimento controllato

```
BEGIN;  
  
INSERT INTO products  
  (product_id, sku, product_name, category, unit_price, active)  
VALUES  
  (1000, 'LAB-SVC', 'Laboratory Service', 'services', 250.00, true)  
RETURNING product_id, sku, product_name, category, unit_price;  
  
ROLLBACK;
```

RETURNING evita di cercare a mano la riga appena inserita.

Esempio: delete controllato

```
BEGIN;  
  
SELECT product_id, sku  
FROM products  
WHERE sku = 'LAB-SVC';  
  
DELETE FROM products  
WHERE sku = 'LAB-SVC'  
RETURNING product_id, sku, product_name;  
  
ROLLBACK;
```

Il DELETE è più sicuro quando il filtro è stato prima provato con una SELECT.

Sequenza consigliata per UPDATE e DELETE

- ❶ Scrivere la SELECT con lo stesso WHERE.
- ❷ Contare le righe candidate.
- ❸ Aprire BEGIN.
- ❹ Eseguire UPDATE o DELETE con RETURNING.
- ❺ Verificare il risultato.
- ❻ Usare ROLLBACK in test, COMMIT solo se la modifica deve diventare definitiva.

Errori frequenti da prevenire

- ▶ eseguire UPDATE o DELETE senza WHERE;
- ▶ non controllare quante righe saranno modificate;
- ▶ confondere test con modifica definitiva;
- ▶ non registrare modifiche business rilevanti;
- ▶ aggiungere vincoli senza verificare i dati esistenti.

5. Pratica guidata

Ripetiamo lo stesso processo su un problema diverso: rappresentazione iniziale, limiti, riorganizzazione, soluzione.

Esercitazione: problema informale

Contesto

Il product manager chiede una nuova tabella per audit prezzi e un aggiornamento controllato di alcuni prodotti. La consegna richiede procedura e controlli, non solo l'istruzione finale.

Esercitazione: specifica corretta

- ▶ input: tabella `products` e categoria scelta;
- ▶ output: tabella `audit`, righe inserite, aggiornamento controllato e verifica;
- ▶ vincolo: tutto deve avvenire dentro una transazione;
- ▶ vincolo: la soluzione deve poter essere annullata con `ROLLBACK` durante il test.

Esercitazione: definizione della soluzione

- ① creare tabella audit con riferimenti al prodotto;
- ② selezionare i prodotti candidati prima dell'update;
- ③ aggiornare con RETURNING;
- ④ scegliere ROLLBACK in laboratorio oppure COMMIT solo dopo verifica.

Implementazione completa: audit e modifica (1/2)

```
SET search_path TO training;

BEGIN;

SELECT product_id, sku, unit_price
FROM products
WHERE category = 'accessories' AND active
ORDER BY product_id;

CREATE TABLE IF NOT EXISTS product_price_audit (
    audit_id bigserial PRIMARY KEY,
    product_id integer NOT NULL REFERENCES products(product_id),
    old_price numeric(10,2) NOT NULL,
    new_price numeric(10,2) NOT NULL,
    reason text NOT NULL,
    changed_at timestamp NOT NULL DEFAULT now()
);
```

Implementazione completa: audit e modifica (2/2)

```
WITH candidates AS (  
    SELECT product_id, unit_price AS old_price,  
           round(unit_price * 1.05, 2) AS new_price  
    FROM products  
    WHERE category = 'accessories' AND active  
) , changed AS (  
    UPDATE products p  
    SET unit_price = c.new_price  
    FROM candidates c  
    WHERE c.product_id = p.product_id  
    RETURNING p.product_id, c.old_price, p.unit_price AS new_price  
)  
INSERT INTO product_price_audit(product_id, old_price, new_price, reason)  
SELECT product_id, old_price, new_price, 'annual price review'  
FROM changed;  
  
SELECT * FROM product_price_audit ORDER BY audit_id DESC;  
  
ROLLBACK;
```

Esercitazione: organizzazione dei 90 minuti

- ▶ 15 min distinguere DDL e DML;
- ▶ 20 min creare tabella audit;
- ▶ 25 min update con pre-controllo e RETURNING;
- ▶ 20 min verifica e rollback;
- ▶ 10 min discussione su commit in produzione.

- ▶ confrontare almeno due soluzioni quando esistono alternative;
- ▶ chiedere quali assunzioni cambierebbero la soluzione;
- ▶ separare errori di sintassi, errori di modello ed errori di interpretazione.

- ▶ una modifica dati senza pre-controllo non è una soluzione didattica accettabile;
- ▶ la transazione rende esplicito il momento in cui ci si assume responsabilità;
- ▶ DDL e DML vanno trattati come parte del modello operativo.